

Lab questions 202509

Question 4 - 1x

Objective

Test the student's ability to identify and reason about web vulnerabilities related to insecure handling of user input, particularly SQL Injection (SQLi), and to apply knowledge of enumeration and exploitation steps in a realistic scenario.

– FROM HERE

Scenario

You are testing a vulnerable e-commerce application. While browsing products, you notice that the website uses a parameterized URL:

`https://shop.example.com/products?category=Pets`

When you insert unexpected input, the server responds with error messages that suggest database queries are being constructed directly from user input.

You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near "Pets" at line X

Questions

1. What vulnerability does this web application have?
2. How would you exploit the vulnerability?

3. What objectives can you fulfill by exploiting the vulnerability?

– TO HERE

Possible Answers

1. SQL Injection

2. Possible exploitation techniques include:

Confirm: a single quote producing a DB error is a reliable sign of injectable input. From there you classify the SQLi type:

Error-based / visible (results appear in the page).

Time-based blind (time delays indicate truth).

Extract data:

We don't have enough information to determine if query results are rendered (besides the error message. If so, we can use a controlled query that returns rows (e.g., a UNION pattern) to read values (columns, version, user table). We would possibly need to iteratively try different numbers of columns, maybe using NULLs or guessing the correct data type (e.g., INTEGER vs VARCHAR).

If results are not rendered, use boolean or time-based techniques to infer characters of strings (one bit/character at a time).

Use enumeration to discover useful targets: database version, current DB user, and table/column names.

Leverage DB capabilities (only if permitted by privileges; may or may not be possible):

Read files accessible to the DB user (e.g., LOAD_FILE() in MySQL) to exfiltrate secrets like configuration files.

If the DB user can write to the webroot (which is rare), write a file (e.g., via SELECT ... INTO OUTFILE) to get remote command execution (web shell).

Use stacked queries (if the server allows multiple statements) to perform additional actions.

3. Typical objectives an attacker could achieve

Data exfiltration: steal user data, credentials, payment info, API keys, DB contents.

Authentication bypass / account takeover: retrieve password hashes or session tokens and impersonate users (including admin).

Privilege escalation / lateral movement: obtain credentials or files that let the attacker access other machines or services.

Remote code execution (RCE) / persistence: if the DB or server permissions allow writing to a web-accessible directory or executing OS commands, the attacker could upload a web shell or execute commands.

Question 5 - 1.5x

Objective

This question assesses students' ability to reason about abusing insecure dynamic library loading in SetUID/SetGID programs (.so injection), understand the attack constraints (runtime loader behavior, privilege model), and explain realistic exploitation and mitigation steps based on the course slides about SetUID/SetGID and shared-object injection.

– FROM HERE

Scenario

A legacy utility, named **vuln**, is installed on a Linux lab machine with the owner root and the set-UID bit set (mode 4755).

The binary provides optional “licensed” functionality by attempting to load a shared library named `./libpremium.so` at runtime using:

```
dlopen(libpremium, RTLD_LAZY)
```

where:

```
#define libpremium "./libpremium.so"
```

If the library cannot be opened, the program falls back to a built-in, non premium, implementation.

The program does not sanitize the working directory or otherwise validate the library contents.

You do not have the source of **vuln**, but you can run it as your unprivileged user and observe its behavior (including with `strace`, e.g., by copying the executable on your attack box), and you can write files into some directories you control in your home directory of the target system.

Questions

1. Under realistic Unix/Linux rules for SetUID binaries, is `LD_PRELOAD` or `LD_LIBRARY_PATH` a reliable way to influence what **vuln** loads? Explain why or why not.
2. What specific vulnerability does **vuln** present? Explain briefly why this is a security problem for a SetUID program.
3. Give a high-level exploitation strategy. In your plan, describe the key characteristics of the exploit.
4. What immediate objectives could an attacker achieve by successfully exploiting this vulnerability?

– TO HERE

Possible Answers

1. About LD_PRELOAD / LD_LIBRARY_PATH and setuid programs

By default most Unix systems ignore LD_PRELOAD and LD_LIBRARY_PATH for SetUID (and SetGID) executables as a deliberate security measure.

Therefore these environment variables are not a reliable vector for attacking a correctly configured setuid binary. The attacker must instead rely on the program loading libraries from paths they control (relative paths, writable directories in a search path, or insecure use of dlopen).

2. Vulnerability identification

vuln performs insecure, relative-path dynamic loading of a shared object (dlopen("./libpremium.so", ...)) while running set-UID root.

This allows a local attacker who can influence the process's working directory or place a file at ./libpremium.so (relative to where the program is launched) to cause the privileged process to load and execute attacker-supplied code.

For setuid programs, this is a severe vulnerability because the injected code runs with the program's elevated privileges.

3. High-level exploitation plan

Malicious .so property: Build a shared library that contains a constructor/initializer (e.g., __attribute__((constructor))) so code runs immediately when the library is loaded, and that code performs the privileged action you want (e.g., escalate privileges and spawn a root shell or write a root-owned file).

The constructor may call `setuid(0)` / `setgid(0)` as needed and then `execve("/bin/sh", ...)` or similar

Placement: Place the malicious `libpremium.so` where `vuln` will actually open `./libpremium.so`. If the program uses a relative path (`./libpremium.so`) then the attacker only needs control of the working directory used when running `vuln` (for example, by creating a writable directory, placing `libpremium.so` there, and invoking `vuln` from that directory).

`dlopen("./libpremium.so", ...)` will attempt to open that specific file path; if it exists and is a valid shared object, the runtime linker will map and run its constructors in the context of the calling process, i.e., with elevated effective UID/GID. That yields code execution under elevated privileges.

4. Attacker objectives

Spawn an interactive root shell (if the attacker can get code executed as root via constructor or exported function).

Read sensitive files (e.g., `/etc/shadow`, configuration files); straightforward once arbitrary code runs as root.

Create persistence (install backdoor/service) or add user accounts.